

Compte rendu du Pianoscalier:

Un escalier transformé en piano

Kaï Djepaxhia / Medelys Seux / Lucille Gontard / Alexis Burgod / Rojhat Yilmaz

Ce projet permet de transformer un escalier ordinaire en un piano géant. Chaque marche détecte le passage d'une personne à l'aide d'un système de barrière laser et déclenche un son spécifique via un module MP3. Une interface web permet de configurer en temps réel la note ou la piste audio associée à chaque marche.

Lien du GitHub: <https://github.com/kai733/pianoscalier>

Lien vidéo de démonstration: <https://www.youtube.com/watch?v=7o9-mmufm38>

Lien vidéo de montage: https://www.youtube.com/watch?v=_22HXT8OAU

Sommaire

- [Fonctionnement](#)
- [Matériel requis](#)
- [Schéma de câblage](#)
- [Installation](#)
- [Lancement](#)
- [Utilisation](#)
- [Explication du code](#)
- [Auteurs](#)

Fonctionnement

Le système repose sur une communication entre une interface web et un Arduino :

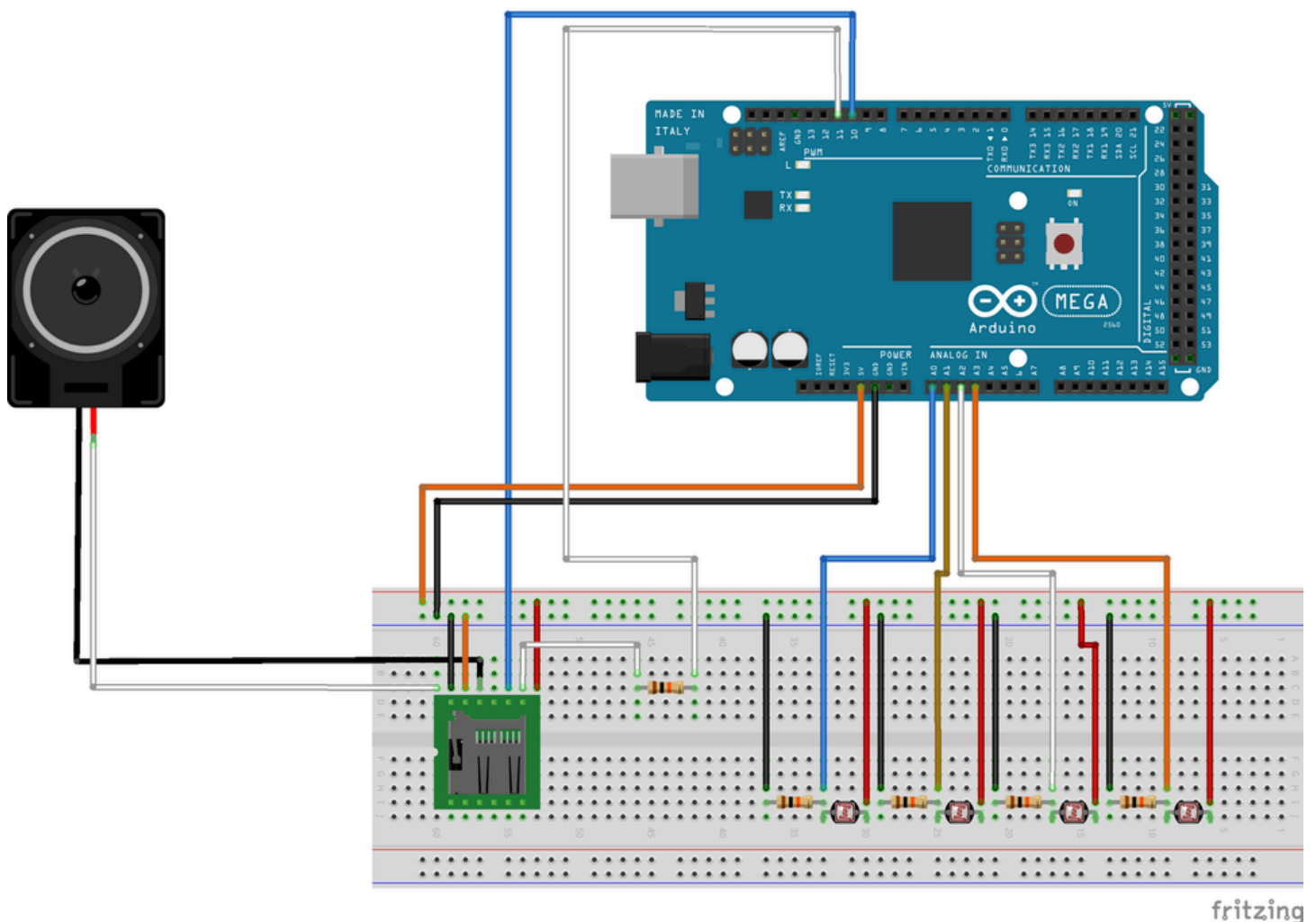
1. Détection : Des photorésistances (LDR) reçoivent un faisceau laser. Lorsqu'une personne pose le pied sur une marche, le faisceau est coupé, la luminosité baisse, et l'Arduino détecte l'événement.
2. Audio : L'Arduino commande un module DFPlayer Mini pour jouer le fichier MP3 correspondant à la marche.
3. Contrôle Web : Une interface HTML/JS communique avec un serveur Node.js via Socket.io. Le serveur transmet les commandes de configuration à l'Arduino par liaison série (USB).

Matériel requis (Pour 4 marches)

- 1 Arduino (Uno ou Mega)
- 1 Module DFPlayer Mini (mp3-tf-16p)
- 4 Photorésistances (LDR)
- 4 Lasers basse consommation
- 4 Résistances de 10k ohms (pour les LDR)
- 1 Résistance de 1k ohm (pour la protection du port RX du DFPlayer)
- 1 Haut-parleur (8 ohms)
- 1 Carte Micro SD (formatée en FAT32)
- Pleins de câbles Arduino

Shéma de cablage (Pour le tutoriel complet voir la vidéo montage)

<https://youtu.be/22HXT80AU>



Installation

Configuration de la carte SD

Les fichiers audio doivent être placés à la racine de la carte SD ou dans un dossier nommé MP3. Ils doivent être nommés avec quatre chiffres :

- 0001.mp3
- 0002.mp3
- 0003.mp3
- etc...

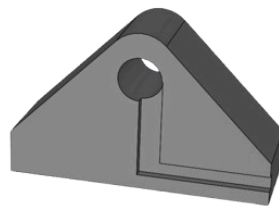
Vous pouvez utiliser les fichiers fournis dans ce projet

Configuration logicielle

1. Téléversez le code Arduino situé dans le dossier arduino/ sur votre carte.
2. Installez Node.js sur votre ordinateur.
3. Dans le dossier du projet, installez les dépendances nécessaires : `npm install`
4. Identifiez le port COM de votre Arduino (ex: COM3 ou `/dev/ttyUSB0`) et modifiez-le dans le fichier `server.js`.

Configuration matérielle

1. Placez une photorésistance par marche. Pour faciliter l'alignement des lasers avec les photorésistances nous avons utilisé l'impression 3D pour faire des supports. Nous avons modélisé avec SolidWorks des supports à partir des mesures de la hauteur des lasers.



2. Placez les lasers à l'opposé, de sorte qu'ils pointent sur les photorésistances
3. Appuyez sur le bouton RESET de la carte Arduino

Lancement

1. Branchez l'Arduino en USB à l'ordinateur.
2. Lancez le serveur Node.js : `node server.js`

3. Ouvrez votre navigateur à l'adresse suivante : <http://localhost:3000>

Utilisation

- Interface : Cliquez sur une marche pour ouvrir le piano virtuel et assigner une nouvelle note.
- Presets : Le bouton Musique 1 configure automatiquement les marches pour la music Rat Dance.

Explication du code

Nous allons ici expliquer et justifier le code que nous avons utilisé pour réaliser ce projet.

1. Firmware (Arduino)

Fichier : *Piano-stairs.ino*

A. Gestion Audio et Communication

Nous utilisons deux ports série distincts pour éviter les conflits de données :

- **Serial (USB - 115200 bauds)** : Dédié à la communication avec l'ordinateur (Node.js) pour recevoir les configurations. La vitesse élevée (115200 vs 9600 standard) réduit la latence de transmission.
- **SoftwareSerial (Pins 10/11 - 9600 bauds)** : Dédié au module MP3 (DFPlayer Mini). Ce module nécessite une vitesse fixe de 9600 bauds.

```
15 void setup() {
16     Serial.begin(115200);
17     mySoftwareSerial.begin(9600);
18     if (!myDFPlayer.begin(mySoftwareSerial)) {
19         while (true)
20             ;
21     }
22     myDFPlayer.volume(30); // Volume du son (Max à 30)
23 }
```

B. Machine à État (State Change Detection)

Pour la détection des pas, nous n'utilisons pas une lecture simple, mais une **détection de changement d'état**.

```

38     if (val <= threshold) { // Si un pied est détecté
39         if (!isPlaying[i]) { // ET qu'on n'était pas déjà en train de jouer
40             myDFPlayer.play(trackMapping[i]); // --> On joue le son UNE SEULE FOIS
41             isPlaying[i] = true; // --> On verrouille l'état
42             Serial.println("STEP_ON:" + String(i + 1));
43         }
44     } else {
45         if (isPlaying[i]) {
46             isPlaying[i] = false; // --> On déverrouille quand le pied part
47         }
48     }
49 }
50 delay(50); // --> Petit délai
51 }

```

Justification :

Si nous avions simplement utilisé `if (val <= threshold) play()`, la fonction `play()` serait appelée des centaines de fois par seconde tant que le pied reste sur la marche, bloquant le module MP3. Cette logique garantit une lecture unique par appui.

C. Configuration Dynamique

Le système permet de changer les sons sans re-téléverser le code Arduino.

```

26     if (Serial.available() > 0) {
27         String data = Serial.readStringUntil('\n'); // Lit la ligne envoyée
28         if (data.startsWith("M")) { // Si ça commence par "M" (ex: M1:05)
29             int stepIdx = data.substring(1, 2).toInt() - 1; // Récupère l'index (0 à 3)
30             int trackNum = data.substring(3).toInt(); // Récupère le numéro de piste
31             if (stepIdx >= 0 && stepIdx < NB_STEPS) {
32                 trackMapping[stepIdx] = trackNum; // Met à jour le tableau
33             }
34         }
35     }

```

- **Protocole** : L'Arduino écoute les chaînes de caractères au format `M{index_marche};{numero_piste}` (ex: M1:5).
- **Parsing** : La fonction `Serial.readStringUntil('\n')` capture la commande et met à jour le tableau `trackMapping[]`.

2. Middleware (Serveur Node.js)

Fichier : *server.js*

Le navigateur web ne pouvant accéder directement aux ports USB de l'ordinateur (pour des raisons de sécurité), ce serveur Node.js sert de d'intermédiaire.

A. Stack Technique

- **serialport** : Gère la connexion physique avec l'Arduino.
- **socket.io** : Gère la connexion temps réel avec l'interface Web.
- **express** : Sert les fichiers statiques (HTML/CSS/JS).

B. Le Pont WebSocket <--> Série

Contrairement au protocole HTTP classique (qui nécessite de recharger la page), **Socket.io** maintient un canal ouvert permanent.

1. **Réception** : Le serveur écoute l'événement `changeNote` venant du Web.
2. **Traitement** : Il convertit l'objet JSON `{ step: 1, track: 5 }` en chaîne optimisée `M1:5\n`.
3. **Envoi** : Il écrit cette chaîne sur le port série vers l'Arduino.

Justification : Cette architecture décentralisée permet de déporter l'intelligence (gestion des presets, interface graphique) sur l'ordinateur, laissant l'Arduino se concentrer sur les capteurs.

3. Interface Utilisateur (Frontend)

Fichier : index.html

L'interface est conçue pour être modulaire et scalable (extensible).

A. Génération Procédurale du DOM

Au lieu d'écrire le code HTML de chaque marche manuellement, le code JavaScript génère les éléments :

```

416 const container = document.getElementById("stairs-wrapper");
417 for (let i = numberOfSteps; i >= 1; i--) {
418     const div = document.createElement("div");
419     div.className = "step-container";
420     div.id = `step-${i}`;
421     div.style.zIndex = i * 10;
422     div.style.marginLeft = `${(i - 1) * offsetPerStep}px`;
423     div.innerHTML = `
424     <div class="step-header" onclick="togglePiano(${i})">
425         <span class="step-title">Marche ${i}</span>
426         <div class="step-note">
427             <span id="note-display-${i}">Do2</span>
428             <span class="arrow">▼</span>
429         </div>
430     </div>
431     <div class="piano-wrapper">${createPianoHTML(i)}</div>`;
432     container.appendChild(div);
433 }

```

- **Avantage :** Si l'escalier passe de 4 à 20 marches, il suffit de changer la variable `const numberOfSteps = 20`. Le code s'adapte automatiquement sans réécriture HTML.

B. Gestion des Notes et Presets

Un dictionnaire `noteToTrack` fait la correspondance entre le langage musical ("Do2", "Ré#2") et le langage machine (Piste 1, Piste 4).


```

352     const noteToTrack = {
353         Do2: 1,
354         "Do#2": 2,
355         Ré2: 3,
356         "Ré#2": 4,
357         Mi2: 5,
358         Fa2: 6,
359         "Fa#2": 7,
360         Sol2: 8,
361         "Sol#2": 9,
362         La2: 10,
363         "La#2": 11,
364         Si2: 12,
365         Do3: 13,
366         "Do#3": 14,
367         Ré3: 15,
368         "Ré#3": 16,
369         Mi3: 17,
370         Fa3: 18,
371         "Fa#3": 19,
372         Sol3: 20,
373         "Sol#3": 21,
374         La3: 22,
375         "La#3": 23,
376         Si3: 24,
377         Do4: 25,
378     };

```

La fonctionnalité "Exemples musique" (Presets) permet d'automatiser la configuration :

- Au clic, le script parcourt un objet de configuration pré-défini.
- Il met à jour visuellement toutes les marches simultanément.

C. Expérience Utilisateur

- **Feedback Visuel** : Lorsqu'une marche est configurée, l'interface affiche clairement la note/piste assignée.

Auteurs

Projet réalisé par Rojhat YILMAZ, Kaï DJEPAXHIA, Alexis BURGOD, Medelys SEUX et Lucille GONTARD.